ARQUITETURA CLOUD-NATIVE PARA SISTEMAS ESCALÁVEIS: INTEGRAÇÃO DE MICROSSERVIÇOS, SERVERLESS E OBSERVABILIDADE
CLOUD-NATIVE ARCHITECTURE FOR SCALABLE SYSTEMS: INTEGRATING MICROSERVICES, SERVERLESS COMPUTING, AND OBSERVABILITY

Gabriel Ribeiro de Camargo Miranda[1]
Discente de Engenharia de Software
Afiliação e contato institucional a confirmar

Deivison Shindi Takatu[2]
Orientador
Afiliação e contato institucional a confirmar

Glauco Todesco[3]
Coorientador
Afiliação e contato institucional a confirmar

RESUMO: Este trabalho propõe uma arquitetura cloud-native para sistemas escaláveis no contexto de pesquisa de Iniciação Científica. A investigação é aplicada, exploratória e qualitativa, com revisão bibliográfica não sistemática e modelagem. Como resultados parciais, o repositório reúne descrição arquitetural, diagramas e matriz de decisões. A proposta combina aplicação web, API Gateway, serviços serverless e em contêineres, persistência relacional e NoSQL, mensageria e observabilidade. Não há protótipo nem medições; as decisões de projeto ainda requerem validação experimental.

Palavras-chave: Elasticidade; Gestão de custos; Persistência poliglota; Sistemas distribuídos; Tolerância a falhas.

ABSTRACT: This paper proposes a cloud-native architecture for scalable systems within an ongoing undergraduate research project. The applied, exploratory, and qualitative study combines a non-systematic literature review with architectural modeling. Partial results documented in the repository include an architecture description, diagrams, and a decision matrix. The proposal combines a web application, API Gateway, serverless and containerized services, relational and NoSQL persistence, messaging, and observability. No prototype or measurements are available; the design decisions require experimental validation.

Keywords: Cost management; Distributed systems; Elasticity; Fault tolerance; Polyglot persistence.

1 INTRODUÇÃO

A ampliação de serviços digitais aumenta a necessidade de sistemas que suportem variações de carga, integração entre componentes e mudanças frequentes.

A adoção de nuvem, por si só, não garante elasticidade, disponibilidade ou evolução independente: essas propriedades dependem de decisões sobre fronteiras dos serviços, comunicação, dados e operação. A literatura sobre microsserviços associa a decomposição adequada à evolução independente, mas também aponta custos de integração, testes e observabilidade distribuída (Newman, 2021; Di Francesco; Lago; Malavolta, 2019).

O problema investigado é como estruturar uma arquitetura em nuvem que favoreça a evolução e a escalabilidade sem transferir a complexidade para a operação, a segurança e o monitoramento. A pesquisa é desenvolvida no contexto de Iniciação Científica e, até o momento, produziu documentação e modelos arquiteturais, sem implementação funcional ou avaliação experimental. Essa condição delimita o alcance das conclusões e torna pertinente apresentar os resultados como parciais.

O objetivo deste trabalho é propor uma arquitetura cloud-native que relacione requisitos não funcionais a componentes, responsabilidades e tecnologias de nuvem. Especificamente, busca-se organizar decisões sobre microsserviços, execução serverless e em contêineres, comunicação assíncrona, persistência, segurança e observabilidade, além de definir uma base para validação futura. A contribuição desta etapa é o conjunto rastreável de descrição arquitetural, diagramas de componentes e fluxos e matriz de decisões existente no repositório.

A questão de pesquisa é: como combinar serviços gerenciados, execução serverless e microsserviços em contêineres de modo a preservar a capacidade de evolução sem comprometer segurança, observabilidade e controle de custos? A pergunta não é tratada como uma comparação universal entre tecnologias. Ela orienta a construção de uma proposta situada, na qual cada decisão deve ser justificada pelo requisito que atende e pela forma como poderá ser verificada posteriormente.

O escopo está concentrado na arquitetura lógica e nos requisitos não funcionais. Não são apresentados resultados de desempenho, custos reais ou evidências de disponibilidade, porque esses dados dependem de uma implementação e de uma infraestrutura de teste ainda não realizadas. A distinção entre decisão de projeto e resultado medido é mantida ao longo do texto para evitar que uma hipótese arquitetural seja apresentada como conclusão experimental.

METODOLOGIA

A pesquisa é aplicada, exploratória e qualitativa. O procedimento combina levantamento bibliográfico não sistemático, análise documental dos materiais do projeto e modelagem de uma proposta arquitetural. Não há protocolo de revisão sistemática registrado; por isso, o levantamento é tratado como exploratório e não pretende esgotar a literatura.

Primeiro, foram reunidas fontes sobre microsserviços, computação serverless, contêineres, segurança e arquitetura em nuvem. Em seguida, os objetivos do projeto foram relacionados a requisitos não funcionais de escalabilidade, disponibilidade, manutenibilidade, segurança, observabilidade e controle de custos. Esses requisitos orientaram a definição de componentes, responsabilidades e alternativas tecnológicas. A arquitetura e os fluxos foram registrados em diagramas Mermaid e em descrição textual, acompanhados por uma matriz de decisão.

A verificação nesta etapa é documental: examina a coerência entre problema, requisitos e decisões registradas. Não foram implementados protótipo, testes de carga ou experimentos com usuários. A validação futura deverá implementar uma prova de conceito e medir, em cenários controlados, latência, throughput, taxa de erro, comportamento sob falhas e custo operacional.

O levantamento bibliográfico foi orientado por quatro eixos: decomposição de sistemas, modelos de execução em nuvem, proteção de serviços distribuídos e observabilidade operacional. Foram priorizados livros técnicos, artigos de mapeamento ou revisão e documentação oficial dos serviços considerados na arquitetura. As fontes foram utilizadas para identificar benefícios, limitações e condições de uso, e não apenas para reunir definições. Quando a documentação de um provedor foi usada, ela foi tratada como referência técnica para a decisão correspondente.

Na etapa de modelagem, os requisitos foram transformados em responsabilidades arquiteturais. A apresentação foi separada do processamento; o ponto de entrada foi separado dos serviços de domínio; e a persistência foi analisada conforme a natureza dos dados. Depois, foram descritos os fluxos síncronos e assíncronos, os limites de confiança e os pontos de observabilidade. O resultado foi

comparado com os requisitos em uma matriz que explicita a verificação ainda necessária.

A análise também considera ameaças à validade. A ausência de protótipo impede afirmar que os componentes terão a latência, a disponibilidade ou o custo esperados. A escolha de serviços AWS reduz a neutralidade da comparação e pode aumentar dependência do provedor. Além disso, a revisão não sistemática pode deixar estudos relevantes fora do conjunto analisado. Essas limitações não invalidam a modelagem, mas delimitam o tipo de conclusão que pode ser apresentada nesta fase.

3 REFERENCIAL TEÓRICO

3.1 Microsserviços e fronteiras de responsabilidade

Microsserviços organizam uma aplicação em serviços com responsabilidades delimitadas e possibilidade de implantação independente. A definição das fronteiras deve refletir capacidades do domínio, evitando apenas distribuir um monólito sem reduzir o acoplamento (Newman, 2021). Um mapeamento sistemático da área também aponta benefícios de evolução e escalabilidade seletivas, junto a desafios de integração, testes e observabilidade (Di Francesco; Lago; Malavolta, 2019).

A decomposição, portanto, não é um objetivo isolado. Um serviço precisa possuir uma responsabilidade compreensível, uma interface estável e um ciclo de evolução que possa ser administrado pela equipe. A divisão excessiva aumenta chamadas remotas e torna transações distribuídas mais difíceis; a divisão insuficiente mantém dependências e impede escalonamento seletivo. A proposta utiliza esse equilíbrio como critério para decidir quais capacidades podem ser executadas em Lambda e quais precisam de um serviço em contêiner.

3.2 Computação serverless e contêineres

Na computação serverless, o provedor administra parcela relevante da infraestrutura e executa funções sob demanda. Revisões da área descrevem aplicações adequadas a eventos e cargas variáveis, mas registram limitações de inicialização, duração, depuração, portabilidade e dependência do provedor (Shafiei;

Khonsari; Mousavi, 2019; John; Gupta, 2021; Li et al., 2021; Wen et al., 2022). Estudos sobre impacto econômico e carregamento de contêineres reforçam que o modelo de execução deve ser escolhido conforme o perfil de cada componente (Adzic; Chatley, 2017; Brooker et al., 2023).

O modelo híbrido é relevante porque os requisitos de uma mesma aplicação não são uniformes. Uma função curta acionada por evento pode se beneficiar da elasticidade e da operação gerenciada do serverless. Um serviço que mantenha processos mais longos, dependa de bibliotecas específicas ou precise de maior controle sobre o runtime pode ser empacotado como contêiner. A decisão precisa considerar duração, memória, frequência de execução, dependências, estado e necessidade de controle operacional.

3.3 Segurança, observabilidade e operação

A comunicação assíncrona por filas e tópicos pode reduzir dependências temporais e absorver variações de demanda, desde que a solução trate duplicidade, idempotência, retentativas e mensagens que não possam ser processadas. Segurança e observabilidade também devem ser consideradas desde o desenho: autenticação, autorização, criptografia, isolamento, logs, métricas e rastreamento precisam acompanhar os fluxos distribuídos (Indrasiri; Siriwardena, 2020; AWS, s.d.-e). Esses princípios orientam a proposta, mas sua efetividade ainda depende de validação no protótipo.

Em uma arquitetura distribuída, o controle de acesso não pode depender somente da interface. O gateway deve validar o contexto da requisição, os serviços devem verificar as permissões necessárias e as funções devem receber apenas as credenciais exigidas por sua responsabilidade. A segmentação de rede, a criptografia em trânsito e em repouso e o registro de operações complementam a autenticação. Essas medidas reduzem a superfície de ataque, mas não substituem revisão de configuração e testes de ameaça.

Observabilidade é tratada como mecanismo de diagnóstico, e não apenas como coleta de logs. Logs estruturados permitem reconstruir uma execução; métricas mostram tendências de latência, erros e consumo; alertas indicam desvios; e

rastreamento distribuído relaciona uma requisição aos componentes percorridos. Sem correlação entre essas evidências, a decomposição em serviços pode dificultar a localização da causa de uma falha.

3.4 Trabalhos relacionados e síntese dos critérios

Os trabalhos de revisão sobre microsserviços e serverless convergem em dois pontos. O primeiro é que modularidade e elasticidade podem reduzir o esforço de evolução em determinados cenários. O segundo é que os benefícios vêm acompanhados de custos de integração, monitoramento, portabilidade e governança. Adzic e Chatley (2017) destacam o impacto econômico e arquitetural do serverless, enquanto Brooker et al. (2023) discutem o carregamento sob demanda de contêineres. Essas contribuições sustentam a decisão de não tratar serverless como substituto universal de contêineres.

Com base nessa síntese, a proposta utiliza cinco critérios: responsabilidade de negócio, perfil de carga, necessidade de estado, criticidade do dado e exigência de diagnóstico. A combinação desses critérios permite justificar a arquitetura com mais precisão do que a simples adoção de uma lista de serviços. Também torna possível construir um protocolo de validação que compare o comportamento esperado com o observado.

4 RESULTADOS E DISCUSSÃO

4.1 Visão geral da solução

Os artefatos do projeto são uma descrição dos componentes, diagramas Mermaid e uma matriz de decisões. A arquitetura proposta inclui aplicação web em S3 e CloudFront, API Gateway e Cognito, processamento em Lambda e ECS/Fargate, PostgreSQL e DynamoDB, mensageria SQS/SNS e telemetria via CloudWatch e rastreamento distribuído (AWS, s.d.-a; AWS, s.d.-b; AWS, s.d.-c; AWS, s.d.-d).

O fluxo começa no navegador do usuário. O conteúdo estático é distribuído pelo CloudFront a partir do S3, enquanto as chamadas de negócio seguem para o API Gateway. O gateway funciona como fronteira de entrada, aplica limites de requisição e encaminha cada operação para a função ou serviço correspondente. O Cognito

concentra a identidade e permite que as chamadas carreguem o contexto de autorização para os componentes internos.

4.2 Componentes e responsabilidades

A proposta prevê Lambda para tarefas reativas e curtas e ECS/Fargate para serviços que demandem maior controle de execução. A persistência relacional ou NoSQL será escolhida conforme os dados e seus padrões de acesso. São hipóteses de projeto, não resultados de comparação empírica.

Na camada de processamento, Lambda atende eventos, integrações e operações de curta duração. ECS/Fargate atende serviços de domínio que tenham ciclo de execução mais longo, dependências que não se ajustem ao modelo de função ou necessidade de manter um processo controlado. Essa separação não significa que os serviços sejam independentes de qualquer contrato: as interfaces precisam ser versionadas e os erros devem possuir comportamento definido para que a evolução não quebre consumidores.

Na camada de dados, o RDS PostgreSQL é indicado para entidades com relacionamentos, consistência transacional e consultas estruturadas. O DynamoDB é indicado para dados cuja chave de acesso seja conhecida e cuja elasticidade seja mais importante que relacionamentos complexos. A chamada persistência poliglota exige governança: cada dado deve ter um sistema de referência, regras para sincronização e política de recuperação. Sem essas regras, o uso de dois bancos apenas deslocaria a complexidade para a aplicação.

4.3 Comunicação e comportamento sob falhas

Operações que exigem resposta imediata podem seguir pelo caminho síncrono entre gateway e serviço. Processamentos demorados, notificações e integrações externas devem ser publicados em tópicos ou filas. O produtor não precisa aguardar a conclusão do consumidor, mas deve registrar um identificador idempotente e tratar a confirmação de entrega. O consumidor deve tolerar mensagens duplicadas, utilizar retentativas controladas e encaminhar mensagens não processáveis para uma fila de análise.

estratégia reduz o acoplamento temporal, mas introduz consistência eventual e exige que o domínio aceite o atraso entre publicação e processamento. Por isso, a decisão por mensageria deve estar associada ao requisito do caso de uso. Quando o usuário precisa de confirmação imediata, o fluxo assíncrono não deve ser ocultado como se fosse transação concluída. A interface pode informar processamento pendente e consultar o estado posteriormente.

4.4 Segurança, escalabilidade e observabilidade

O acesso público é protegido por HTTPS, gateway e políticas de autenticação. Recursos de persistência não devem ser expostos diretamente à Internet; funções e contêineres devem utilizar permissões de menor privilégio e, quando necessário, sub-redes privadas. Os registros de execução devem remover segredos e dados pessoais desnecessários. A proposta considera essas medidas como requisitos de implantação, não como propriedades automaticamente garantidas pela escolha da AWS.

A elasticidade é obtida em níveis diferentes. Lambda e DynamoDB podem acompanhar variações de demanda segundo sua configuração de capacidade. ECS/Fargate pode aumentar o número de tarefas conforme métricas de utilização e fila. O RDS requer planejamento de capacidade, réplicas e limites de conexão, porque a escalabilidade do banco relacional não é idêntica à escalabilidade das funções. O CloudWatch e o X-Ray conectam esse comportamento a indicadores de operação, permitindo observar se o aumento de capacidade realmente reduz filas e erros.

4.5 Matriz de decisões

O Quadro 1 relaciona requisitos, decisões e verificações futuras. Como a distribuição de componentes pode ampliar a complexidade de integração e operação, a avaliação deverá considerar segurança e custo além de escalabilidade (Di Francesco; Lago; Malavolta, 2019; AWS, s.d.-e).

Quadro 1 – Requisitos, decisões arquiteturais e validação prevista

Requisito	Decisão proposta	Verificação futura
Escalabilidade	Lambda e ECS/Fargate com expansão conforme a carga	Medir latência, throughput, erros e consumo de recursos em cenários controlados.

Segurança	Cognito, autorização por tokens, TLS, IAM e isolamento de rede	Revisar configuração, permissões e ameaças do fluxo entre componentes.
Persistência	RDS PostgreSQL e DynamoDB conforme o padrão de dados	Avaliar consistência, concorrência, integridade e adequação aos acessos.
Observabilidade	Logs estruturados, métricas, alertas e rastreamento	Injetar falhas e verificar detecção e rastreabilidade ponta a ponta.
Resiliência	SQS/SNS, retentativas, backoff e idempotência	Exercitar duplicidade, reprocessamento e mensagens não processáveis.

Fonte: Elaborado pelos autores com base nos artefatos de arquitetura do projeto.

Não há protótipo, dados experimentais ou medições de desempenho. Portanto, não é possível afirmar redução de latência, aumento de throughput, economia ou melhoria de disponibilidade. O resultado desta etapa é explicitar decisões e riscos para orientar implementação e avaliação.

4.6 Plano de validação e limitações

A próxima etapa deve implementar um protótipo mínimo com pelo menos um fluxo síncrono, um fluxo assíncrono, uma função Lambda, um serviço em contêiner e os dois padrões de persistência. O experimento deve comparar uma carga nominal e cenários de pico, registrar latência nos percentis P50, P95 e P99, taxa de erro, throughput, tempo de processamento da fila e consumo de recursos. Também devem ser provocadas falhas controladas para verificar retentativas, idempotência, alertas e rastreabilidade.

Os resultados deverão ser interpretados com cautela. Uma redução de latência em um cenário sintético não prova superioridade geral; uma configuração barata em baixa carga pode ser inadequada em picos; e uma arquitetura que suporta falhas pode ainda exigir recuperação manual. O protocolo de avaliação deve registrar a configuração, o volume de requisições, o tempo de execução, os custos e as limitações para que a comparação seja reproduzível.

5 CONSIDERAÇÕES FINAIS

A pesquisa organizou uma proposta cloud-native e documentou seus componentes, fluxos e decisões. Sua contribuição nesta etapa é um artefato conceitual rastreável para orientar a implementação e planejar a avaliação.

Sem protótipo e validação empírica, benefícios de escalabilidade, resiliência e custo permanecem hipóteses. Os próximos passos são revisar a arquitetura com os orientadores, implementar uma prova de conceito e medir seu comportamento sob carga e falhas. Os resultados deverão orientar os ajustes da proposta.

O trabalho também evidencia que a arquitetura deve ser analisada como um conjunto de compromissos. Microsserviços favorecem limites de evolução, mas ampliam o número de interações. Serverless reduz a administração de servidores, mas pode aumentar dependência de provedor e exigir atenção a inicialização e limites de execução. A persistência híbrida atende perfis diferentes de dados, mas exige governança. A decisão final deve ser tomada a partir do problema e da evidência, e não da adoção automática de uma tecnologia.

Como contribuição acadêmica, a pesquisa deixa uma relação explícita entre problema, requisitos, decisões, riscos e verificações futuras. Como limitação, ainda não há dados coletados que permitam confirmar os benefícios esperados. A continuidade recomendada é realizar a prova de conceito, registrar o protocolo experimental, revisar a arquitetura com os orientadores e atualizar o manuscrito com resultados que possam ser auditados.

REFERÊNCIAS BIBLIOGRÁFICAS

ADZIC, G.; CHATLEY, R. Serverless computing: economic and architectural impact. In: ESEC/FSE 2017. New York: ACM, 2017. p. 884-889. DOI: 10.1145/3106237.3117767.

AWS. Amazon API Gateway Developer Guide. Disponível em: https://docs.aws.amazon.com/apigateway/. Acesso em: 17 set. 2026. (s.d.-a).

AWS. Amazon CloudFront Developer Guide. Disponível em: https://docs.aws.amazon.com/cloudfront/. Acesso em: 17 set. 2026. (s.d.-b).

AWS. Amazon Cognito Developer Guide. Disponível em: https://docs.aws.amazon.com/cognito/. Acesso em: 17 set. 2026. (s.d.-c).

AWS. AWS Lambda Developer Guide. Disponível em: https://docs.aws.amazon.com/lambda/. Acesso em: 17 set. 2026. (s.d.-d).

AWS. AWS Well-Architected Framework. Disponível em: https://docs.aws.amazon.com/wellarchitected/. Acesso em: 17 set. 2026. (s.d.-e).

BROOKER, M. et al. On-demand container loading in AWS Lambda. arXiv, 2023. Disponível em: https://arxiv.org/abs/2305.13162. Acesso em: 17 set. 2026.

DI FRANCESCO, P.; LAGO, P.; MALAVOLTA, I. Architecting with microservices: a systematic mapping study. The Journal of Systems and Software, v. 150, p. 77-97,

2019. DOI: 10.1016/j.jss.2019.01.001.
INDRASIRI, K.; SIRIWARDENA, P. Microservices Security in Action. Shelter Island: Manning Publications, 2020.
JOHN, J.; GUPTA, S. A survey on serverless computing. arXiv, 2021. Disponível em: https://arxiv.org/abs/2106.11773. Acesso em: 17 set. 2026.
LI, Z. et al. The serverless computing survey: a technical primer for design architecture. arXiv, 2021. Disponível em: https://arxiv.org/abs/2112.12921. Acesso em: 17 set. 2026.
NEWMAN, S. Building Microservices: Designing Fine-Grained Systems. 2. ed. Sebastopol: O'Reilly Media, 2021.
SHAFIEI, H.; KHONSARI, A.; MOUSAVI, P. Serverless computing: a survey of opportunities, challenges and applications. arXiv, 2019. Disponível em: https://arxiv.org/abs/1911.01296. Acesso em: 17 set. 2026.
WEN, J. et al. Rise of the planet of serverless computing: a systematic review. arXiv, 2022. Disponível em: https://arxiv.org/abs/2206.12275. Acesso em: 17 set. 2026.